

Brand Strategy Document

Abhishek Prakash — Personal Professional Brand

Engineer who ships — backend to AI

Unique Value Proposition

Most engineers own one layer. They're either the backend person, the DevOps person, or the AI person. I've never had that luxury — and I'm better for it.

At Dassault SolidWorks, I led a full microservices migration while also building the observability stack to monitor it. At Deloitte, I architected the event pipeline and wrote the frontend that consumed it. At Zomato, I improved search at the infrastructure level and in the UI layer simultaneously.

I build systems end-to-end — distributed backends, cloud infrastructure, and AI-powered features — and I've done it in production environments serving millions of users. That's the difference between someone who can do it and someone who has done it.

Brand Pillars

1. End-to-End Ownership I don't hand off problems — I follow them. From database schema to CI/CD pipeline to production monitoring, I stay with a system until it works. This isn't stubbornness; it's how good software gets built.

2. Performance at Scale Numbers tell the story: 85% API latency reduction. 1M+ monthly transactions at 99.9% uptime. 50K daily users. These aren't estimates from a pitch deck — they're from systems I built and maintained in production.

3. AI-First Thinking Generative AI isn't something I bolt onto existing projects to make them look current. I understand how RAG pipelines work, how to architect LangChain-based systems, and how to deploy them reliably. AI is part of how I think about building, not an afterthought.

4. Relentless Shipping Intern at Zomato. Full-time developer at Deloitte. Co-op at Dassault SolidWorks. Co-founder of Swadh Food Delivery. MS student at Northeastern. I don't wait for the right moment — I build during it.

5. Human Reliability I mentor 200+ MS students through code reviews and system design sessions. I've led cross-functional teams and co-founded a company with four people. Technical skill matters, but so does being someone your team can count on. I take both seriously.

Target Audience Personas

Persona 1 — The Engineering Hiring Manager

Who they are: Senior Engineering Manager or VP of Engineering at a fintech company, SaaS platform, or enterprise software company. They've been burned before by engineers who were great in interviews but couldn't own a problem in production. They're not looking for someone to assign tickets to — they need someone who can take a system, understand it, and make it better.

What they care about: Real production experience. Engineers who don't need hand-holding. People who can debug at 2am without escalating everything. Proof that you've shipped, not just built.

Why I'm the answer: I've operated in PCI-DSS compliant fintech environments, migrated legacy enterprise systems without downtime, and built observability tools so teams can actually see what's happening. My work has served real users at real scale. That's not a school project — that's a track record.

How I reach them: LinkedIn posts about production lessons learned. Resume metrics that speak their language. A GitHub that shows I actually commit code.

Persona 2 — The AI-Forward Tech Lead

Who they are: Staff Engineer or CTO at an AI-first startup or ML-powered product company. They're building something genuinely new and they need engineers who understand both sides — the intelligence layer and the infrastructure underneath it. They're tired of ML engineers who can't write a reliable API and backend engineers who treat AI as a black box.

What they care about: Engineers who can build a RAG pipeline and deploy it. People who understand vector databases, embeddings, and LangChain but can also write production-grade FastAPI backends. Full-stack AI — not just model wrappers.

Why I'm the answer: I built an LLM Document Intelligence System with GPT-4, LangChain, FastAPI, and RAG architecture that achieved 85% Q&A accuracy on unstructured documents. And I deployed it. That combination — AI knowledge plus backend rigor plus infrastructure ownership — is exactly what they're looking for.

How I reach them: GitHub repos with strong READMEs that explain the architecture. Posts about AI system design. A website that shows projects with real outcomes, not just tech stacks.

Competitive Positioning Statement

There are plenty of engineers graduating with strong academic backgrounds and side projects. There are also experienced professionals with deep specialization in one domain. I occupy a different position: a builder with real production experience across backend systems, cloud infrastructure, and applied AI — who has shipped products at companies like Dassault SolidWorks, Deloitte, and Zomato, and who co-founded a company from scratch.

I'm not the most senior person in the room. But I'm the one who has already done the work that most candidates are still talking about doing.

My differentiation is simple: I build things that work, at scale, in production. That's harder to fake than a resume — and mine comes with the metrics to prove it.

Career Positioning Strategy

Target roles: Software Engineer II/III (Backend), Full-Stack Engineer, Applied AI/ML Engineer, Forward Deployed Engineer, Platform Engineer

Target companies: Stripe, Plaid, Coinbase, Wise, MongoDB, Anthropic, Figma, Vercel, Bloomberg, Robinhood, Datadog

Core message for every application: I've built production systems serving millions of users. I own problems end-to-end. I bring AI fluency to backend engineering. I'm available May 2026.

What I lead with for backend roles: The SolidWorks latency story (3s → 450ms, 85% reduction) and the Deloitte pipeline (1M+ monthly transactions, zero message loss).

What I lead with for AI roles: The LLM Document Intelligence System (GPT-4, LangChain, RAG, 85% accuracy) and my ability to deploy AI systems, not just prototype them.

Section 3 — Brand Identity System

Logo & Logotype

My personal brand mark is a geometric square containing the initials **"AP"** in bold white on an electric blue (`#1E6FFF`) background. The logotype pairs this mark with "Abhishek Prakash" in Space Grotesk Bold and "Engineer who ships — backend to AI" in Inter Regular below it.

Logo variations:

- Primary: AP mark + full name + tagline
- Compact: AP mark + name only
- Minimal: AP mark alone (for favicon, social avatars)

Color Palette

Color	Hex Code	Usage
Deep Navy	<code>#0A1628</code>	Primary background
Card Surface	<code>#0F2040</code>	Cards, panels
Electric Blue	<code>#1E6FFF</code>	Primary accent, CTAs
Sky Blue	<code>#4D91FF</code>	Highlights, hover states
Pure White	<code>#FFFFFF</code>	Primary text
Muted Silver	<code>#A0AEC0</code>	Secondary text

Typography System

Primary font — Space Grotesk Used for: Name, headings, section titles, brand mark Why: Technical, modern, geometric — feels like it belongs in an engineering context without being cold.

Secondary font — Inter Used for: Body text, bullet points, descriptions, captions Why: Extremely legible at small sizes, widely used in professional SaaS products, pairs cleanly with Space Grotesk.

Brand Voice

- **Direct** — Lead with the metric, not the adjective
- **Precise** — "85% latency reduction" not "significantly faster"
- **Confident** — Builders don't apologize for their work
- **Curious** — Always learning, always shipping

Section 4 — Brand Stories

Brand Story 1 — The Hero's Journey

Narrative Framework: The Hero's Journey *Why I chose it: My career path is genuinely non-linear — Mechanical Engineering to software to AI. The Hero's Journey captures that transformation better than a straight chronological summary.*

I didn't start as a software engineer.

I started as a Mechanical Engineering student at VIT University, learning how to design systems that work under pressure — systems where failure has consequences. I spent four years learning to think like an engineer: identify the constraint, isolate the variable, fix the root cause.

Then I wrote my first real piece of code, and something clicked. The constraints were different, but the thinking was identical.

My call to adventure came in the form of an internship at Zomato. Fifty thousand daily users. Real latency problems. Real consequences if search broke on a Friday night. I wasn't ready — but I shipped anyway. I learned that software engineering is just engineering with different materials.

The trials came at Deloitte. A fintech platform processing one million transactions a month. PCI-DSS compliance. RabbitMQ connecting eight microservices across billing, payment, and customer systems. Systems that could not fail. I built them anyway. They held.

The transformation happened at Dassault SolidWorks. I didn't just write features — I led a migration. Four legacy applications, ten thousand enterprise users, and a mandate to eliminate deployment outages without taking anything offline. I designed the architecture, built the observability layer, and shipped it. API latency dropped 85%. Release cycles went from bi-weekly to weekly. Nothing broke.

Along the way, I co-founded Swadh Food Delivery — took a product from a whiteboard to two thousand active users across Vellore. I learned that ownership isn't a job title. It's a decision.

Now I'm completing my MS at Northeastern, graduating May 2026. Full-stack, backend, AI/ML — with real production experience to back it up.

The ordinary world was a Mechanical Engineering classroom. The return is an engineer who builds things that hold up.

Brand Story 2 — The Build

Narrative Framework: Before / After / Because *Why I chose it: This framework is best for demonstrating concrete transformation — showing not just what I did, but what changed because of it. It maps directly to how hiring managers evaluate impact.*

Before

When I joined Dassault SolidWorks for my co-op, four legacy applications were running on Perl and C#. Deployment was a twice-a-month ritual that everyone dreaded. Engineers were afraid to push changes because something always broke in production. Ten thousand enterprise users were experiencing outages they shouldn't have had to tolerate. The system worked — barely — and nobody wanted to touch it.

After

Eight months later, all four applications had been migrated to TypeScript microservices running on Kubernetes. We had a centralized REST service layer with Swagger-defined contracts. We had observability pipelines — Fluent Bit feeding into Elasticsearch — so every engineer could see exactly what was happening across every service in real time. We had an internal NPM package that cut microservice integration time by 40% for every team that came after us. Releases went from bi-weekly to weekly. API latency dropped from three seconds to 450 milliseconds — an 85% reduction. Deployment outages were eliminated.

Ten thousand enterprise users never noticed the migration. That's the point.

Because

Because I didn't treat it as a ticket. I treated it as a system problem that needed an owner. I mapped the dependencies before touching a line of code. I built the observability layer first so we could see what we were changing. I created the NPM package not just for this migration but for every engineer who would work on this platform after me.

The lesson I carry into every role: the difference between a developer and an engineer is ownership. Developers complete tasks. Engineers own outcomes.

That's the standard I hold myself to — and it's the standard I'll bring to whatever I build next.

Section 5 — Social Media Posts

Platform Strategy

Primary platform: LinkedIn Target audience: Engineering hiring managers, tech leads, senior engineers, recruiters at fintech and AI companies. Posting cadence: 2-3 times per week. Content mix: Production lessons learned, architecture breakdowns, job search reflections, project highlights.

Secondary platform: GitHub Every pinned repository has a README that opens with the business problem, not the tech stack. Show the thinking, not just the code.

5 Example LinkedIn Posts

Post 1 — Production Lesson

We reduced API latency by 85% at SolidWorks — from 3 seconds to 450ms.

Here's what actually made the difference (it wasn't what I expected):

The easy answer is "we built a centralized REST service layer." And yes, that's what we did.

But the real answer is that we mapped every dependency before writing a single line of new code.

Four legacy applications. Years of accumulated coupling. Nobody had a complete picture of what called what.

So we built the map first. Then we built the service layer. Then latency dropped 85%.

The technical solution was straightforward. The discipline to understand the problem before solving it — that was the hard part.

That's the lesson I take into every system I work on now.

Post 2 — Career Story

I have a Mechanical Engineering degree.

I'm a software engineer.

People find this confusing. I don't.

Engineering is engineering. The materials change. The thinking doesn't.

At VIT I learned to design systems under constraints. To identify failure modes. To think about what happens when things go wrong before they go wrong.

I use that every day — when I'm designing a distributed pipeline, when I'm planning a migration, when I'm building an AI system that needs to be reliable at scale.

The degree isn't in spite of my career. It's part of why I work the way I do.

Post 3 — AI/Technical

Most people build AI features. I learned to build AI systems.

There's a difference.

An AI feature is a ChatGPT API call wrapped in a try/catch block.

An AI system is a RAG pipeline with chunking strategy, vector store selection, retrieval tuning, a FastAPI backend that handles failures gracefully, and a deployment that doesn't break at 2am.

I built the latter for my LLM Document Intelligence project — GPT-4, LangChain, vector embeddings, 85% answer accuracy on unstructured documents.

The AI part took a week. Making it production-ready took three more.

That's the part nobody talks about.

Post 4 — Entrepreneurship

In 2020 I co-founded a food delivery startup with three other people and zero funding.

We onboarded 100+ restaurants. We grew to 2,000+ active users across Vellore. We built the entire platform ourselves.

I learned more in those months about what it means to own a product than in any course or internship.

When you're the engineer AND the product manager AND the customer support AND the person getting the 2am calls when the app goes down — you develop a different relationship with reliability.

Every system I've built since then, I ask the same question: what happens when this breaks at the worst possible time?

Because I know from experience — it will.

Post 5 — Job Search / Availability

Graduating from Northeastern in May 2026.

MS in Computer Software Engineering. 2.5 years of production experience. Distributed systems, full-stack engineering, applied AI.

What I'm looking for: A team that takes engineering seriously. Problems that are genuinely hard. A codebase I can be proud of contributing to.

What I bring: Systems that I've built and maintained at scale. An end-to-end mindset. The habit of owning outcomes, not just completing tasks.

Open to: Backend, Full-Stack, Applied AI/ML, Platform Engineering roles.

Based in Boston. Open to relocation or remote.

If you're building something interesting — let's talk. prakash.ab@northeastern.edu